

CAPSTONE PROJECT
MATA KULIAH PENGANTAR PEMROGRAMAN
Semester Genap 2025/2026

Pembangunan Aplikasi Web
Web Portofolio Dinamis dengan Python Flask

Mata Kuliah	Pengantar Pemrograman
Sifat Tugas	Individu
Bahasa / Framework	Python 3 & Flask
Metode Submit	Upload ke LMS / e-learning

A. Deskripsi Proyek

Mahasiswa diminta untuk membangun sebuah aplikasi web portofolio yang bersifat dinamis menggunakan bahasa pemrograman Python dengan framework Flask. Aplikasi terdiri dari dua bagian utama: halaman publik yang menampilkan portofolio kepada pengunjung, dan halaman dashboard (admin) yang memungkinkan pemilik untuk menambah, mengubah, serta menghapus konten portofolio tanpa perlu mengubah kode program.

Proyek ini merupakan puncak dari seluruh materi yang telah dipelajari selama satu semester, mulai dari dasar Python (variabel, kondisi, perulangan, fungsi), hingga penerapannya dalam pengembangan web nyata menggunakan Flask, Jinja2, dan SQLite. Mahasiswa diharapkan mampu mengintegrasikan semua konsep tersebut dalam satu aplikasi yang fungsional, rapi, dan dapat digunakan secara nyata.

B. Tujuan Pembelajaran

Setelah menyelesaikan capstone project ini, mahasiswa diharapkan mampu:

- Menerapkan konsep dasar Python (tipe data, kondisi, perulangan, fungsi, list, dictionary) dalam konteks pengembangan web.
- Membangun aplikasi web sederhana menggunakan framework Flask (routing, template, form handling).

- Menggunakan template engine Jinja2 untuk merender halaman HTML secara dinamis dari data Python.
- Menyimpan dan mengambil data menggunakan database SQLite melalui Flask-SQLAlchemy atau modul sqlite3.
- Mengimplementasikan fitur upload file (gambar) dan menyimpannya di server.
- Menerapkan autentikasi sederhana (login/logout) untuk melindungi halaman dashboard.
- Mendeploy aplikasi Python ke layanan hosting dan mendokumentasikan proyek secara tertulis.

C. Spesifikasi Fitur Aplikasi

C.1 Halaman Publik (Frontend Portofolio)

Halaman publik dapat diakses oleh siapa saja tanpa login. Wajib memuat halaman-halaman berikut:

No	Halaman	Konten Wajib
1	Home / Beranda	Foto profil, nama, headline/tagline profesi, ringkasan singkat (about), dan tombol navigasi ke bagian lain.
2	About	Deskripsi lengkap diri, pendidikan, dan daftar skill/keahlian (ditampilkan dinamis dari database).
3	Portofolio	Daftar semua karya/proyek yang tersimpan di database. Setiap item menampilkan: gambar thumbnail, judul proyek, deskripsi singkat, teknologi yang digunakan, dan link proyek/GitHub.
4	Detail Proyek	Halaman detail untuk setiap proyek (diarahkan dari halaman Portofolio). Menampilkan informasi lengkap proyek.
5	Kontak	Form kontak (nama, email, pesan) yang tersimpan ke database, dan informasi kontak pemilik (email, GitHub, LinkedIn).

C.2 Halaman Dashboard Admin (Backend)

Halaman dashboard hanya dapat diakses setelah login. Wajib memuat fitur-fitur berikut:

No	Fitur	Detail Fungsionalitas
1	Login & Logout	Form login dengan username dan password. Session management menggunakan Flask session. Semua halaman dashboard dilindungi (redirect ke login jika belum masuk).

2	Ringkasan Dashboard	Halaman utama dashboard menampilkan statistik: jumlah total proyek, jumlah pesan masuk yang belum dibaca, dan shortcut ke fitur-fitur utama.
3	Manajemen Proyek (CRUD)	Create: Form untuk menambah proyek baru (judul, deskripsi, teknologi, link, upload gambar). Read: Tabel daftar semua proyek. Update: Form edit data proyek yang sudah ada. Delete: Hapus proyek dengan konfirmasi.
4	Upload Gambar Proyek	Mendukung upload file gambar (PNG, JPG, JPEG, GIF, WEBP). Validasi tipe file di sisi server. Gambar disimpan di folder static/uploads/. Ukuran file maksimal 5 MB.
5	Manajemen Profil	Form untuk mengubah data profil: nama, headline, deskripsi about, foto profil, daftar skill (ditambah/dihapus secara dinamis). Data disimpan ke database.
6	Kotak Masuk Pesan	Menampilkan semua pesan dari form kontak. Setiap pesan dapat ditandai 'sudah dibaca' atau dihapus.

C.3 Ketentuan Teknis

Backend	Python 3 + Flask (wajib). Boleh menggunakan ekstensi seperti Flask-SQLAlchemy, Flask-Login, Flask-WTF.
Database	SQLite (wajib). File database disimpan di dalam folder project (misal: portfolio.db).
Template Engine	Jinja2 (sudah termasuk dalam Flask). Wajib menggunakan template inheritance (base.html).
Frontend	HTML5, CSS3, JavaScript. Boleh menggunakan Bootstrap atau Tailwind CSS untuk styling.
Struktur Folder	Mengikuti konvensi Flask: app.py / run.py di root, folder templates/, folder static/ (css/, js/, uploads/), folder models/ atau models.py.
Version Control	Seluruh kode dikelola dengan Git dan di-push ke repository GitHub yang bersifat public.
Hosting	Aplikasi di-deploy dan dapat diakses secara online menggunakan domain .my.id. Rekomendasi: PythonAnywhere, Railway, atau Render (semua mendukung Flask).

D. Struktur Folder Project yang Disarankan

Berikut adalah contoh struktur folder minimal yang diharapkan:

```

portofolio-flask/
├── app.py                # Entry point aplikasi Flask
├── config.py             # Konfigurasi (SECRET_KEY, database URI,
dll.)
├── models.py             # Definisi model database (Project, Message,
Profile, Skill)
├── portfolio.db          # File database SQLite (di-gitignore)
├── requirements.txt      # Daftar dependensi Python
├── .gitignore            # Minimal: *.db,  pycache  /, .env
├── README.md             # Panduan instalasi dan penggunaan
├── templates/
│   └── base.html         # Template induk (navbar, footer, CSS/JS
links)
│   ├── index.html        # Halaman beranda
│   ├── about.html        # Halaman about
│   ├── portfolio.html    # Halaman daftar portofolio
│   ├── project_detail.html # Halaman detail proyek
│   ├── contact.html      # Halaman kontak
│   └── dashboard/
│       ├── login.html
│       ├── index.html    # Halaman utama dashboard
│       ├── projects.html # Daftar proyek di dashboard
│       ├── add_project.html
│       ├── edit_project.html
│       ├── profile.html  # Edit profil
│       └── messages.html # Kotak masuk pesan
└── static/
    ├── css/
    │   └── style.css
    ├── js/
    │   └── main.js
    └── uploads/          # Gambar yang di-upload (di-gitignore)

```

E. Panduan Teknis (Referensi Implementasi)

Bagian ini adalah panduan konseptual, bukan solusi lengkap. Mahasiswa wajib mengimplementasikan sendiri.

E.1 Inisialisasi Aplikasi Flask (app.py)

Berikut adalah kerangka dasar app.py yang dapat dijadikan titik awal:

```

from flask import Flask, render_template, request, redirect, url_for,
session
from flask_sqlalchemy import SQLAlchemy
import os

app = Flask(  name  )
app.config['SECRET_KEY'] = 'kunci-rahasia-anda'  # Ganti dengan nilai
acak yang kuat
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///portfolio.db'

```

```

app.config['UPLOAD_FOLDER'] = 'static/uploads'
app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024 # Batas 5 MB

db = SQLAlchemy(app)

# Import model dan route setelah db dibuat
from models import Project, Message, Profile, Skill

if __name__ == '__main__':
    with app.app_context():
        db.create_all() # Buat tabel jika belum ada
        app.run(debug=True)

```

E.2 Contoh Definisi Model (models.py)

Berikut contoh sebagian model database menggunakan Flask-SQLAlchemy:

```

from app import db
from datetime import datetime

class Project(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(150), nullable=False)
    description = db.Column(db.Text, nullable=False)
    technologies = db.Column(db.String(200)) # Simpan sebagai string,
    pisah koma
    image_file = db.Column(db.String(120), default='default.jpg')
    github_link = db.Column(db.String(200))
    live_link = db.Column(db.String(200))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

class Message(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), nullable=False)
    message = db.Column(db.Text, nullable=False)
    is_read = db.Column(db.Boolean, default=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

```

E.3 Contoh Route CRUD Proyek

Berikut contoh kerangka route untuk operasi CRUD pada proyek:

```

import os
from werkzeug.utils import secure_filename

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'webp'}

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

```

```

# Tambah proyek baru
@app.route('/dashboard/projects/add', methods=['GET', 'POST'])
def add_project():
    if 'user' not in session:          # Cek login
        return redirect(url_for('login'))
    if request.method == 'POST':
        title = request.form.get('title')
        desc = request.form.get('description')
        tech = request.form.get('technologies')
        file = request.files.get('image')
        img_name = 'default.jpg'
        if file and allowed_file(file.filename):
            img_name = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'],
img_name))
        project = Project(title=title, description=desc,
                           technologies=tech, image_file=img_name)
        db.session.add(project)
        db.session.commit()
        return redirect(url_for('dashboard_projects'))
    return render_template('dashboard/add_project.html')

# Hapus proyek
@app.route('/dashboard/projects/delete/<int:id>', methods=['POST'])
def delete_project(id):
    if 'user' not in session:
        return redirect(url_for('login'))
    project = Project.query.get_or_404(id)
    db.session.delete(project)
    db.session.commit()
    return redirect(url_for('dashboard_projects'))

```

E.4 Contoh Template Jinja2 (portfolio.html)

Berikut contoh penggunaan template inheritance dan looping di Jinja2:

```

{% extends 'base.html' %}

{% block title %}Portofolio{% endblock %}

{% block content %}
<div class='portfolio-grid'>
    {% for project in projects %}
    <div class='project-card'>
        <img src='{{ url_for("static", filename="uploads/" +
project.image_file) }}'
            alt='{{ project.title }}'>
        <h3>{{ project.title }}</h3>
        <p>{{ project.description[:120] }}...</p>
        <div class='tech-tags'>
            {% for tech in project.technologies.split(',') %}
            <span class='tag'>{{ tech.strip() }}</span>
            {% endfor %}
        </div>
    </div>
    </div>

```

```

    <a href='{{ url_for("project_detail", id=project.id) }}'>Lihat
Detail</a>
</div>
{% else %}
    <p>Belum ada proyek. Tambahkan dari dashboard.</p>
{% endfor %}
</div>
{% endblock %}

```

F. Ketentuan Pengumpulan

File yang dikumpulkan terdiri dari 5 (lima) komponen wajib berikut:

No	Komponen	Keterangan	Format
1	Source Code	Seluruh source code project Python Flask yang telah selesai. Tidak menyertakan file .db, folder <code>__pycache__</code> , dan folder uploads/. Wajib menyertakan file requirements.txt dan README.md.	ZIP (juga via GitHub)
2	Aplikasi Web Live	Aplikasi yang sudah di-deploy dan dapat diakses online menggunakan domain .my.id. Link URL wajib dicantumkan di laporan.	Link URL (di laporan)
3	Repository GitHub	Repository GitHub public yang berisi seluruh source code. Wajib memiliki commit history (minimal 5 commit) yang mencerminkan proses pengerjaan secara bertahap. Link wajib dicantumkan di laporan.	Link GitHub (di laporan)
4	Video Demo & Penjelasan	Video yang di-upload ke YouTube (tidak di-private). Konten video: (1) demo seluruh fitur halaman publik, (2) demo login dan semua fitur dashboard, (3) penjelasan singkat struktur kode Flask (routing, model, template). Durasi minimal 10 menit. Link wajib dicantumkan di laporan.	Link YouTube (di laporan)
5	Laporan Project	Laporan lengkap menggunakan template yang disediakan pada bagian Lampiran soal ini. Laporan memuat semua link, screenshot aplikasi, dan penjelasan implementasi.	DOCX / PDF

Catatan: Akun demo untuk login dashboard wajib dicantumkan di laporan (username dan password) agar dosen dapat menguji fitur dashboard secara langsung.

G. Ketentuan Akademik

- Tugas dikerjakan secara individu. Kesamaan kode yang signifikan antar mahasiswa akan mengakibatkan nilai 0 bagi semua pihak yang terlibat.
- Penggunaan template CSS/UI (Bootstrap, Tailwind) diperbolehkan dan tidak dianggap plagiarisme. Yang dinilai adalah logika Python/Flask-nya.
- Penggunaan AI generatif untuk menghasilkan keseluruhan kode tidak diperbolehkan. Mahasiswa harus mampu menjelaskan setiap baris kode miliknya.
- Repository GitHub wajib memiliki commit history yang mencerminkan proses pengerjaan bertahap. Satu commit tunggal dengan semua kode akan dicurigai sebagai hasil plagiarisme.
- Akun demo dashboard (username & password) wajib dicantumkan di laporan agar dosen dapat menguji aplikasi secara langsung.
- Pengumpulan melebihi batas waktu akan dikenakan pengurangan nilai 10 poin per hari keterlambatan.

LAMPIRAN
TEMPLATE LAPORAN CAPSTONE PROJECT
(Gunakan template ini sebagai laporan tugas Anda)

LAPORAN CAPSTONE PROJECT
PENGANTAR PEMROGRAMAN
Web Portofolio Dinamis dengan Python Flask

Nama	
NIM	
Kelas	
Mata Kuliah	Pengantar Pemrograman
Dosen Pengampu	
Tahun Akademik	

BAB I PENDAHULUAN

1.1 Latar Belakang

[Jelaskan pentingnya portofolio digital bagi seorang mahasiswa/profesional di bidang teknologi. Mengapa sebuah web portofolio yang dinamis (dapat dikelola tanpa mengubah kode) lebih unggul dibandingkan portofolio statis? Kaitkan dengan materi Python dan Flask yang telah dipelajari. Minimal 2 paragraf.]

1.2 Tujuan

[Tuliskan tujuan dibuatnya aplikasi web portofolio dinamis ini, sesuaikan dengan tujuan pembelajaran pada soal.]

1.3 Manfaat

[Tuliskan manfaat aplikasi, baik bagi Anda sebagai pemilik portofolio maupun sebagai pengunjung yang ingin melihat karya Anda.]

1.4 Ruang Lingkup Proyek

[Tuliskan batasan proyek ini: fitur apa saja yang diimplementasikan dan fitur apa yang tidak termasuk (misal: tidak ada sistem registrasi pengguna baru, tidak ada fitur komentar pengunjung, dll.).]

BAB II TINJAUAN PUSTAKA

2.1 Python sebagai Bahasa Pemrograman Web

[Jelaskan Python sebagai bahasa pemrograman dan keunggulannya untuk pengembangan web. Sebutkan ekosistem library/framework yang mendukung.]

2.2 Framework Flask

[Jelaskan apa itu Flask, perbedaannya dengan Django, konsep microframework, dan komponen utamanya: Routing, Template Engine (Jinja2), WSGI, dan Development Server.]

2.3 SQLite dan Flask-SQLAlchemy

[Jelaskan SQLite sebagai database ringan berbasis file, dan bagaimana Flask-SQLAlchemy (ORM) mempermudah interaksi dengan database dari kode Python tanpa menulis SQL mentah.]

2.4 Template Engine Jinja2

[Jelaskan Jinja2: template inheritance, variabel (`{{ }}`), control flow (`{% % %}`), dan filter. Berikan contoh penggunaannya dalam konteks proyek ini.]

2.5 Konsep CRUD

[Jelaskan konsep Create, Read, Update, Delete (CRUD) dan bagaimana konsep ini diimplementasikan dalam proyek ini untuk mengelola data portofolio.]

BAB III ANALISIS DAN PERANCANGAN

3.1 Analisis Kebutuhan

[Jelaskan kebutuhan fungsional (apa yang bisa dilakukan sistem) dan non-fungsional (keamanan, performa, kemudahan penggunaan) dari aplikasi ini.]

3.2 Perancangan Database

[Jelaskan desain database Anda. Tabel apa saja yang Anda buat? Apa kolom-kolomnya? Sertakan diagram sederhana atau tabel penjelasan seperti berikut:]

Tabel	Kolom	Keterangan
Project	id, title, description, technologies, image_file, github_link, live_link, created_at	[Isi dengan penjelasan tabel dan kegunaannya]
Message	id, name, email, message, is_read, created_at	[Isi dengan penjelasan]
Profile	[Sesuaikan dengan desain Anda]	[Isi dengan penjelasan]
Skill	[Sesuaikan dengan desain Anda]	[Isi dengan penjelasan]

3.3 Perancangan Arsitektur Aplikasi

[Jelaskan alur kerja aplikasi secara umum: bagaimana request dari browser diproses oleh Flask, diambil datanya dari database, dirender ke template, lalu dikirim kembali ke browser. Sertakan diagram alur sederhana jika memungkinkan.]

3.4 Perancangan Antarmuka (Wireframe/Mockup)

[Sertakan sketsa atau mockup wireframe untuk halaman-halaman utama: Beranda, Portofolio, dan Dashboard. Dapat berupa foto sketsa tangan atau desain digital sederhana.]

BAB IV IMPLEMENTASI

4.1 Lingkungan Pengembangan

[Jelaskan tools yang Anda gunakan: OS, code editor (VSCode, PyCharm, dll.), versi Python, versi Flask, dan library lain yang tertera di requirements.txt.]

4.2 Implementasi Model Database

[Sertakan screenshot kode models.py Anda dan jelaskan setiap model yang didefinisikan, termasuk relasi antar tabel jika ada.]

4.3 Implementasi Routing (app.py)

[Jelaskan routing yang Anda buat. Sertakan screenshot/potongan kode untuk:]

- [Route halaman publik (index, portfolio, project_detail, contact)]
- [Route autentikasi (login, logout) dan mekanisme session]
- [Route dashboard CRUD proyek (tambah, edit, hapus)]
- [Route upload gambar beserta validasinya]

4.4 Implementasi Template Jinja2

[Jelaskan bagaimana Anda menggunakan template inheritance (base.html). Sertakan screenshot template utama dan contoh halaman yang menggunakannya.]

4.5 Implementasi Fitur Utama Lainnya

[Jelaskan implementasi fitur-fitur spesifik yang menjadi keunggulan aplikasi Anda, misalnya: filter/sorting proyek, flash messages, konfirmasi hapus, atau fitur tambahan lain.]

BAB V HASIL DAN PENGUJIAN

5.1 Tampilan Aplikasi

[Sertakan screenshot tampilan akhir aplikasi. Wajib meliputi:]

- [Screenshot halaman Beranda]
- [Screenshot halaman Portofolio (menampilkan minimal 2 proyek dari database)]
- [Screenshot halaman Detail Proyek]
- [Screenshot halaman Kontak]

- [Screenshot halaman Login Dashboard]
- [Screenshot halaman utama Dashboard (dengan statistik)]
- [Screenshot form tambah proyek baru]
- [Screenshot daftar proyek di dashboard]
- [Screenshot kotak masuk pesan]

5.2 Pengujian Fungsionalitas

[Isi tabel pengujian berikut berdasarkan hasil uji yang Anda lakukan:]

No	Skenario Uji	Hasil yang Diharapkan	Hasil Aktual	Status
1	Login dengan username & password yang benar	Diarahkan ke halaman dashboard	[Isi hasil]	[✓/X]
2	Login dengan password yang salah	Muncul pesan error, tidak masuk dashboard	[Isi hasil]	[✓/X]
3	Tambah proyek baru dengan upload gambar	Proyek tersimpan ke database dan gambar tersimpan di folder uploads/	[Isi hasil]	[✓/X]
4	Edit data proyek yang sudah ada	Data terbaru tersimpan dan terlihat di halaman publik	[Isi hasil]	[✓/X]
5	Hapus proyek	Proyek hilang dari database dan halaman portofolio	[Isi hasil]	[✓/X]
6	Akses halaman dashboard tanpa login	Diarahkan ke halaman login	[Isi hasil]	[✓/X]
7	Kirim form kontak dari halaman publik	Pesan tersimpan ke database dan muncul di kotak masuk dashboard	[Isi hasil]	[✓/X]
8	Upload file bukan gambar (misal: .pdf)	Muncul pesan error, file tidak tersimpan	[Isi hasil]	[✓/X]

5.3 Kendala dan Solusi

[Tuliskan minimal 3 kendala teknis yang Anda temui selama pengerjaan proyek ini dan solusi yang Anda terapkan. Contoh: masalah saat konfigurasi upload file, error saat deploy, bug pada query database, dll.]

BAB VI KESIMPULAN DAN SARAN

6.1 Kesimpulan

[Tuliskan kesimpulan dari keseluruhan proyek. Apakah semua fitur berhasil diimplementasikan? Konsep Python/Flask apa yang paling banyak diterapkan? Apa pelajaran terpenting dari proyek ini?]

6.2 Saran Pengembangan

[Tuliskan fitur-fitur yang dapat ditambahkan di masa depan untuk meningkatkan aplikasi, misalnya: dukungan multi-user, integrasi API pihak ketiga, notifikasi email, mode gelap, dll.]

DAFTAR PUSTAKA

[Format penulisan mengikuti aturan dosen. Contoh referensi yang relevan:]

Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

Ronacher, A. (2024). Flask Documentation. Pallets Projects.
<https://flask.palletsprojects.com/>

SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>

[Tambahkan semua referensi lain yang Anda gunakan: tutorial, artikel, Stack Overflow, dokumentasi library, dll.]

LAMPIRAN

Lampiran 1: Data Akun Demo

Informasi berikut diperlukan agar dosen dapat menguji fitur dashboard secara langsung:

URL Aplikasi (.my.id)	[Isi dengan URL lengkap aplikasi]
Username Dashboard	[Isi dengan username admin]
Password Dashboard	[Isi dengan password admin]
Link GitHub Repository	[Isi dengan URL repository GitHub]
Link Video YouTube	[Isi dengan URL video demo]

Lampiran 2: Daftar Library (requirements.txt)

[Salin isi file requirements.txt Anda ke sini:]

```
Flask==3.0.0
Flask-SQLAlchemy==3.1.1
Werkzeug==3.0.1
# Tambahkan library lain yang Anda gunakan...
```

--- Akhir Template Laporan ---